

# **A Technical Blueprint for a Privacy-Preserving Cookie Analysis Browser Extension**

## **Section 1: Architecting a Modern Browser Extension with Manifest V3**

The development of browser extensions has undergone a significant architectural evolution with the introduction of Manifest V3 (MV3). This new standard, enforced by major browser vendors like Google and Mozilla, represents a fundamental shift away from the persistent, resource-intensive models of the past toward a more secure, performant, and privacy-conscious framework.<sup>1</sup> For a project that handles sensitive data like browser cookies and communicates with external services, a deep understanding of the MV3 paradigm is not merely a best practice but a foundational requirement.

The core philosophy of MV3 is to enhance user safety and browser efficiency by moving extensions to an event-driven architecture. This design minimizes the extension's resource footprint by ensuring that its components are only active when explicitly needed. This section deconstructs this modern architecture, detailing the roles of its primary components and the communication mechanisms that bind them, providing the essential blueprint for the proposed cookie analysis tool.

### **1.1. The Manifest V3 Paradigm Shift: From Persistence to Events**

The transition from Manifest V2 to V3 is characterized by a move from long-lived background pages to ephemeral service workers. This change has profound implications for how extensions are designed, forcing a shift towards a stateless, event-driven model that prioritizes efficiency and security above the convenience of

persistent state.<sup>1</sup>

### 1.1.1. The manifest.json File: The Extension's Blueprint

At the heart of every browser extension is the manifest.json file. This is not merely a metadata file but the central configuration hub and security policy enforcer for the entire application.<sup>3</sup> It explicitly declares the extension's name, version, description, icons, and, most importantly, the permissions it requires to function. For this project, the key declarations will include:

- **manifest\_version:** Must be set to 3 to opt into the modern MV3 platform.<sup>3</sup>
- **name, version, description:** Standard metadata for identifying the extension.<sup>3</sup>
- **permissions:** An array of strings requesting access to specific extension APIs. For this project, this will critically include "cookies" and "storage".<sup>6</sup>
- **host\_permissions:** An array of match patterns specifying which web origins the extension can interact with. To fetch all cookies and communicate with an external API, this will require broad permissions.<sup>7</sup>
- **background:** An object that registers the service worker file (e.g., "service\_worker": "background.js").<sup>3</sup>
- **action:** Defines the extension's icon in the browser toolbar and the HTML file for its popup user interface (e.g., "default\_popup": "popup.html").<sup>9</sup>

### 1.1.2. The Service Worker: The Event-Driven Core

The most significant architectural change in MV3 is the replacement of the persistent background page with an event-driven service worker.<sup>1</sup> Unlike its predecessor, which ran continuously from the moment the browser started, the service worker has a carefully managed, ephemeral lifecycle.<sup>4</sup> It is initialized only when needed—in response to events such as a user clicking the toolbar icon, a message from another part of the extension, a registered alarm firing, or a specific browser event like a cookie change.<sup>11</sup>

Once its task is complete and it enters a period of inactivity, the browser will aggressively terminate the service worker to conserve system resources.<sup>3</sup> This behavior necessitates a fundamental shift in architectural thinking. Developers can no

longer rely on global variables or in-memory state to persist data between events, as this state is wiped clean each time the worker is terminated. Consequently, the extension's core logic must be designed to be stateless. Any data or state that needs to persist across events must be explicitly saved to a durable storage medium, for which the

chrome.storage API is the designated tool. This event-driven, stateless model is more complex than the persistent model of MV2 but yields a more robust, efficient, and secure extension.

### **1.1.3. Content Scripts: The Bridge to the Web Page**

Content scripts are JavaScript files that run in the context of a web page, not the extension itself.<sup>9</sup> They operate within a sandboxed, "isolated world," which means they can access and manipulate the Document Object Model (DOM) of the page but cannot see or interact with the JavaScript variables and functions defined by the page's own scripts.<sup>13</sup> This isolation is a key security feature, preventing conflicts and potential exploits from the web page's code.

While powerful for interacting with web content, content scripts have only limited access to the rich set of extension APIs.<sup>13</sup> For this project, their role is secondary. They would be used if the analysis process needed to be triggered by a user interaction directly on a webpage (e.g., right-clicking an element). In such a scenario, the content script would act as a trigger, sending a message to the service worker to initiate the primary logic.

### **1.1.4. The Popup and User Interface (UI) Components**

The primary user interface for this extension will be the popup, defined by an HTML file (popup.html) and its corresponding JavaScript (popup.js).<sup>3</sup> This UI appears when the user clicks the extension's icon in the browser toolbar. It is the ideal place for displaying the analyzed cookie data and providing controls for initiating scans or configuring settings.

It is crucial to recognize that the popup's environment is also ephemeral. The popup

and its scripts are loaded only when it is opened and are immediately unmounted and destroyed the moment it loses focus or is closed.<sup>4</sup> Therefore, the popup script should be treated as a transient view controller. It is responsible for rendering the UI and dispatching user actions (as messages) to the service worker, but it should never be used to manage long-term state or perform ongoing background tasks. All substantive work must be delegated to the service worker.

## 1.2. The Lifeline of MV3: Inter-Component Communication

Given the strict isolation between the service worker, content scripts, and the popup, a robust communication mechanism is essential. Manifest V3 relies on an asynchronous message passing system to connect these disparate components.<sup>4</sup> The

`chrome.runtime.sendMessage()` and `chrome.runtime.onMessage` APIs are not merely features but the fundamental communication bus that enables the entire architecture to function cohesively.<sup>13</sup>

A typical data flow for this project would follow this event-driven pattern:

1. A user clicks a "Fetch Cookies" button within the `popup.js` script.
2. The `popup.js` script calls `chrome.runtime.sendMessage({ command: "fetchAllCookies" })`.
3. The service worker (`background.js`), which has an active `chrome.runtime.onMessage.addListener(...)` listener, receives this message.
4. The listener's callback function executes, triggering the `chrome.cookies.getAll()` method to begin the data collection process.
5. Once the data is collected and processed, the service worker can send a response back to the popup or store the results in `chrome.storage` for the popup to retrieve.

This clear, decoupled pattern ensures that each component adheres to its designated role, respecting the architectural principles of Manifest V3.

The "limitations" often associated with MV3, such as the prohibition of remotely hosted code and the deprecation of the powerful but risky blocking version of the `webRequest` API, are not arbitrary restrictions. They are deliberate, security-driven design choices.<sup>1</sup> MV2 allowed extensions to fetch and execute JavaScript from a remote server, creating a significant security loophole where unreviewed, malicious

code could be introduced after an extension was published.<sup>1</sup> Similarly, the old `webRequest` API forced extensions to act as a proxy for all network traffic, introducing performance bottlenecks and major privacy concerns.<sup>1</sup> MV3 closes these vectors by mandating that all executable code be bundled within the extension package and by replacing the old API with the more constrained `declarativeNetRequest` API.<sup>1</sup> This security-first philosophy dictates that all aspects of this project, particularly its communication with the external ML API, must be designed to operate within these stricter, safer boundaries.

---

**Table 1.1: Comparison of Extension Components in Manifest V3**

| Component             | Execution Context                | DOM Access                                | Extension API Access                    | Lifespan                                  | Primary Use Case for this Project                                                                          |
|-----------------------|----------------------------------|-------------------------------------------|-----------------------------------------|-------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Service Worker</b> | Separate background thread       | None                                      | Full access (with permissions)          | Event-driven; terminated when idle        | Core logic: fetching cookies, communicating with external APIs, managing storage, handling PIR operations. |
| <b>Content Script</b> | Isolated world within a web page | Full access to the page's DOM             | Limited subset (e.g., runtime, storage) | Tied to the life of the web page          | Secondary; could be used to trigger analysis from a specific webpage context if needed.                    |
| <b>Popup Script</b>   | Extension's popup document       | Access to its own <code>popup.html</code> | Full access (with permissions)          | Ephemeral; exists only while the popup is | User interface: displaying results,                                                                        |

|  |  |     |  |      |                                                                                 |
|--|--|-----|--|------|---------------------------------------------------------------------------------|
|  |  | DOM |  | open | providing buttons to initiate analysis, sending messages to the service worker. |
|--|--|-----|--|------|---------------------------------------------------------------------------------|

---

## Section 2: Comprehensive Cookie Interrogation via the Extensions API

To perform a meaningful analysis, the extension must first gain comprehensive access to the browser's cookie data. The `chrome.cookies` API provides the necessary tools for this task, allowing for both a complete snapshot of all existing cookies and real-time monitoring of any changes.<sup>18</sup> This section provides a masterclass on leveraging this API, detailing the required permissions, the methods for data retrieval, and the rich metadata that can be extracted for machine learning analysis.

### 2.1. Gaining Access: Permissions and Prerequisites

Before any code can be executed, the extension must declare its intent to access cookie data in the `manifest.json` file. This is a cornerstone of the extension security model, ensuring transparency and user consent.

#### 2.1.1. Manifest Declarations

Two distinct types of permissions are mandatory for the core functionality of this project:

1. **API Permission:** The "cookies" permission must be included in the permissions array. This grants the extension access to the `chrome.cookies` namespace and its

methods. Without this, any attempt to call the API will fail.<sup>6</sup>

2. **Host Permissions:** Access to the API is not sufficient on its own. The extension's visibility into the cookie store is governed by the host permissions model. To read cookies for a specific website (e.g., example.com), the extension must be granted permission for that origin in the host\_permissions array.<sup>7</sup>

This creates a critical causal link: the "cookies" permission unlocks the tool, but host\_permissions defines where that tool can be used. For a developer tool designed to analyze *all* cookies across all websites, a broad host permission is required. The most comprehensive pattern is "<all\_urls>", which grants access to cookies from any HTTP or HTTPS origin.<sup>22</sup>

An example manifest.json configuration would look like this:

JSON

```
{
  "name": "Cookie Analyzer",
  "manifest_version": 3,
  "version": "1.0",
  "permissions": [
    "cookies",
    "storage"
  ],
  "host_permissions": [
    "<all_urls>"
  ],
  "background": {
    "service_worker": "background.js"
  },
  "action": {
    "default_popup": "popup.html"
  }
}
```

### 2.1.2. User Consent and Warnings

Requesting broad permissions like "<all\_urls>" has significant security implications. During installation, Chrome will present the user with a warning dialogue explaining that the extension can "Read and change all your data on all websites".<sup>6</sup> For a publicly distributed extension, such a warning can deter users. However, for an internal development tool where the users are trusted developers aware of its purpose, this level of access is both necessary and acceptable.

## 2.2. The `chrome.cookies.getAll()` Method: A Complete Snapshot

The primary method for collecting cookie data is `chrome.cookies.getAll()`. This asynchronous function retrieves all cookies that match a specified set of criteria, returning them as an array of Cookie objects.<sup>23</sup>

### 2.2.1. Function Signature and Parameters

The function is called as `chrome.cookies.getAll(details, callback)`, where `details` is an object used to filter the results. To fetch every cookie stored in the browser, an empty `details` object is passed: `{}`. The `details` object can filter cookies by numerous properties, including domain, name, path, secure, session, and `storeId`.<sup>18</sup>

### 2.2.2. Practical Implementation in the Service Worker

The following code sample demonstrates how to call `getAll()` from the service worker and process the results. Note the use of modern promise-based syntax with `async/await`, which is fully supported in MV3 service workers.



## JavaScript

```
// background.js

// Listener for messages from the popup
chrome.runtime.onMessage.addListener((message, sender, sendResponse) => {
  if (message.command === "fetchAllCookies") {
    getAllCookies().then(sendResponse);
    return true; // Indicates that the response is sent asynchronously
  }
});

async function getAllCookies() {
  try {
    const cookies = await chrome.cookies.getAll({});
    console.log(`Found ${cookies.length} cookies.`);

    // Format the cookies for analysis
    const formattedCookies = cookies.map(cookie => {
      return {
        name: cookie.name,
        value: cookie.value,
        domain: cookie.domain,
        path: cookie.path,
        expires: cookie.expirationDate |

        | 'Session',
        httpOnly: cookie.httpOnly,
        secure: cookie.secure,
        sameSite: cookie.sameSite,
        storeId: cookie.storeId
      };
    });

    return { success: true, data: formattedCookies };
  } catch (error) {
    console.error("Error fetching cookies:", error);
    return { success: false, error: error.message };
  }
}
```

```
}  
}
```

### 2.2.3. Understanding Cookie Stores

The `storeId` property of a cookie is an important piece of metadata. It identifies the cookie store that the cookie belongs to. Browsers use different stores to isolate cookies, most commonly between the main user profile and private or incognito windows.<sup>18</sup> The

`chrome.cookies.getAllCookieStores()` method can be used to list all available stores and their associated tab IDs, allowing for a more granular and complete analysis that respects user context.<sup>22</sup>

## 2.3. The `chrome.cookies.onChanged` Listener: Real-Time Monitoring

In addition to fetching a static snapshot, the extension can monitor cookie changes in real time using the `chrome.cookies.onChanged` event listener. This is a powerful feature for dynamic analysis, allowing the extension to react instantly when a cookie is created, updated, or deleted.<sup>26</sup>

### 2.3.1. Event-Driven Updates

By adding a listener in the service worker, the extension can be woken from an idle state whenever a relevant cookie event occurs.

```
JavaScript
```

```
// background.js
```

```
chrome.cookies.onChanged.addListener((changeInfo) => {
  console.log("A cookie has changed!");

  const cause = changeInfo.cause;
  const isRemoved = changeInfo.removed;
  const cookie = changeInfo.cookie;

  if (isRemoved) {
    console.log(`Cookie removed: ${cookie.name} from ${cookie.domain}. Cause: ${cause}`);
  } else {
    console.log(`Cookie set/updated: ${cookie.name} at ${cookie.domain}. Cause: ${cause}`);
  }

  // Here, you could send this update to the ML service for real-time analysis
});
```

### 2.3.2. Interpreting the changeInfo Object

The callback function receives a single changeInfo object that provides full context for the event <sup>27</sup>:

- **removed**: A boolean that is true if the cookie was deleted and false if it was created or overwritten.
- **cookie**: The full Cookie object that was affected.
- **cause**: A string detailing the reason for the change. Common values include 'explicit' (set or removed by a direct API call), 'overwrite' (an existing cookie was replaced), 'expired' (the cookie's expiration date passed), and 'evicted' (removed by the browser due to storage limits).<sup>26</sup>

This real-time data stream can be invaluable for understanding the dynamic behavior of cookies on modern web applications.

## 2.4. The Richness of Cookie Attributes for ML Analysis

The ultimate goal is to analyze cookies using a machine learning model. The effectiveness of this model will depend heavily on the quality of the features it is trained on. While the name and value of a cookie are important, the true analytical power lies in the rich metadata provided by its attributes. Each attribute offers a strong signal about the cookie's intended purpose, security posture, and potential privacy implications.

For instance, a cookie with `httpOnly` set to `true` is inaccessible to client-side JavaScript, a common characteristic of session tokens used for authentication to mitigate cross-site scripting (XSS) attacks.<sup>29</sup> The

`secure` attribute mandates that the cookie only be sent over HTTPS, indicating a need for confidentiality.<sup>30</sup> The

`sameSite` attribute ('Strict', 'Lax', or 'None') is a critical defense against Cross-Site Request Forgery (CSRF) attacks and a primary indicator of whether a cookie is for first-party or third-party contexts.<sup>31</sup> A persistent

`expirationDate` far in the future, combined with `sameSite`: 'None', is a hallmark of a cross-site tracking cookie.

Therefore, the data payload sent to the ML model must be a comprehensive representation of the Cookie object, not just a simple key-value pair. The attributes themselves are the most potent features for classification and analysis. The `Priority` attribute, which may be visible in Chrome DevTools, is a deprecated feature based on an expired IETF draft and should be considered a low-signal or unreliable feature for analysis.<sup>33</sup>

**Table 2.1: Comprehensive Analysis of HTTP Cookie Attributes**

| Attribute Name | Source   | Purpose                        | Example Value | Relevance to ML Analysis                                                                         |
|----------------|----------|--------------------------------|---------------|--------------------------------------------------------------------------------------------------|
| <b>Name</b>    | RFC 6265 | The identifier for the cookie. | session-id    | Feature for identifying common cookie types (e.g., <code>_ga</code> , <code>JSESSIONID</code> ). |

|                          |          |                                                          |                     |                                                                                                    |
|--------------------------|----------|----------------------------------------------------------|---------------------|----------------------------------------------------------------------------------------------------|
| <b>Value</b>             | RFC 6265 | The data stored by the cookie.                           | a3fWa...            | Can be analyzed for entropy, format (e.g., JWT), or PII, but is often opaque.                      |
| <b>Domain</b>            | RFC 6265 | Specifies hosts to which the cookie will be sent.        | .example.com        | Indicates scope. A leading dot signifies it's valid for subdomains, a key tracking feature.        |
| <b>Path</b>              | RFC 6265 | Specifies a URL path that must exist in the request.     | / or /app           | Defines the scope within a domain. A path of / is very broad.                                      |
| <b>Expires / Max-Age</b> | RFC 6265 | Defines the cookie's lifetime.                           | Thu, 21 Oct 2021... | Distinguishes persistent cookies (potential trackers) from session cookies (die on browser close). |
| <b>HttpOnly</b>          | RFC 6265 | If present, the cookie is not accessible via JavaScript. | true                | Strong signal for a security-sensitive cookie, such as an authentication token.                    |
| <b>Secure</b>            | RFC 6265 | If present, the cookie is only sent over HTTPS.          | true                | Indicates the cookie contains sensitive information requiring a secure channel.                    |

|                    |                  |                                                             |                   |                                                                                                                                              |
|--------------------|------------------|-------------------------------------------------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SameSite</b>    | RFC 6265bis      | Controls whether a cookie is sent with cross-site requests. | Strict, Lax, None | Crucial for security and privacy. 'None' is required for third-party contexts and is a strong indicator of a tracking or advertising cookie. |
| <b>Partitioned</b> | CHIPS            | Isolates third-party cookies per top-level site.            | true              | A modern privacy feature to mitigate cross-site tracking by partitioning state.                                                              |
| <b>Priority</b>    | Deprecated Draft | (Deprecated) Suggested a priority for cookie eviction.      | High, Medium, Low | Low signal. A historical artifact that indicates a site was trying to influence eviction, but is no longer standard.                         |

---

## Section 3: Secure Communication with External Analysis Services

Once cookie data is collected, it must be transmitted to the external ML service for analysis. This phase of the operation is fraught with security risks. A compromised communication channel or mishandled credentials could expose sensitive user data or lead to abuse of the analysis API. This section establishes a secure architecture for this communication, focusing on the non-negotiable requirement of protecting API keys and enforcing strict network policies.

### 3.1. The Anti-Pattern: Why You Must Not Store API Keys Client-Side

The most critical security principle in this project is that **API keys and other secrets must never be stored in the client-side extension code**. Embedding a private API key directly into a JavaScript file or placing it in `chrome.storage` is a severe security vulnerability that will inevitably lead to its compromise.<sup>36</sup>

#### 3.1.1. Inherent Risks of Client-Side Storage

A browser extension, despite the sandboxing provided by MV3, is fundamentally a client-side application. Its entire package—including all JavaScript, HTML, and CSS files—is downloaded to the user's machine. A motivated individual can easily locate the extension's files on their local disk, unpack the package, and inspect the source code in plain text. Any hardcoded key will be immediately visible.<sup>37</sup>

#### 3.1.2. `chrome.storage` is Not a Vault

It is a common misconception that `chrome.storage` provides a secure, encrypted vault for secrets. This is incorrect. Both `chrome.storage.local` and `chrome.storage.sync` are unencrypted key-value stores designed for persisting application state and user settings, not credentials.<sup>38</sup> Data stored here can be accessed by any part of the extension with the

storage permission and can be read by anyone with access to the user's local browser profile files. Storing an API key in `chrome.storage` offers no meaningful protection over hardcoding it.

### 3.2. The Recommended Architecture: The Server-Side Proxy Pattern

The only robust and secure method for managing the ML service's API key is to keep it

off the client entirely. This is achieved by implementing a server-side proxy pattern.<sup>36</sup>

### 3.2.1. Architectural Overview

In this architecture, the browser extension does not communicate directly with the third-party ML service. Instead, it communicates with a trusted intermediary server—a proxy—that is under the developer's control. The workflow is as follows:

1. The extension collects the cookie data.
2. The extension sends a fetch request containing the cookie data to the developer's proxy server. This request does *not* contain the API key.
3. The proxy server receives the request from the extension.
4. The proxy server securely retrieves the ML service API key from its own secure storage (e.g., environment variables, a secret manager).
5. The proxy server appends the API key to the request and forwards it to the actual ML service endpoint.
6. The ML service processes the request and sends the analysis results back to the proxy.
7. The proxy forwards the results back to the browser extension.

### 3.2.2. Benefits of the Proxy Pattern

This pattern provides numerous security and operational benefits:

- **Credential Isolation:** The API key never leaves the secure server environment, completely eliminating the risk of client-side exposure.<sup>36</sup>
- **Centralized Management:** API keys can be rotated, updated, or revoked on the server without needing to update and republish the browser extension.<sup>37</sup>
- **Enhanced Security Controls:** The proxy server can implement additional security layers, such as rate limiting to prevent abuse, IP address filtering, and detailed logging for auditing purposes.<sup>41</sup>
- **Abstraction:** The extension's logic is simplified, as it only needs to know about a single, trusted endpoint (the proxy).

This architecture is not an optional enhancement; it is a mandatory component for any



extension that needs to use a private API key securely.

### 3.3. Implementing Secure API Calls from the Service Worker

With the proxy architecture in place, the service worker can be configured to make secure network requests. This requires proper manifest declarations to handle cross-origin requests and a strict Content Security Policy to lock down network capabilities.

#### 3.3.1. Using the fetch API with Host Permissions

The service worker will use the standard fetch API to send data to the proxy server. To permit this cross-origin request, the proxy server's URL must be explicitly declared in the `host_permissions` field of `manifest.json`.<sup>40</sup> This declaration informs the browser that the extension is authorized to initiate communication with that specific origin, satisfying the browser's cross-origin security model for extensions.<sup>44</sup>

JavaScript

// background.js

```
async function sendCookiesForAnalysis(cookieData) {  
  const PROXY_URL = 'https://api.your-secure-proxy.com/analyze-cookies';  
  
  try {  
    const response = await fetch(PROXY_URL, {  
      method: 'POST',  
      headers: {  
        'Content-Type': 'application/json',  
      },  
      body: JSON.stringify(cookieData)  
    });  
  }  
};
```

```

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const analysisResult = await response.json();
    return analysisResult;
  } catch (error) {
    console.error("Error sending data to proxy:", error);
    throw error;
  }
}

```

### 3.3.2. Content Security Policy (CSP) for Defense-in-Depth

host\_permissions and content\_security\_policy are two sides of the same security coin, providing a layered defense for network requests. While host\_permissions grants the *capability* to connect to an origin, the CSP *enforces* a stricter policy on what resources can be loaded or connected to.

By defining a content\_security\_policy in the manifest, the developer can instruct the browser to block any outgoing connections that are not explicitly allowed. The connect-src directive is particularly important; it should be configured to allow connections only to the extension's own origin ('self') and the trusted proxy server's origin.

JSON

```

{
  "manifest_version": 3,
  "name": "Cookie Analyzer",
  ...
  "host_permissions": [
    "<all_urls>",

```

```

    "https://api.your-secure-proxy.com/"
  ],
  "content_security_policy": {
    "extension_pages": "script-src 'self'; object-src 'self'; connect-src 'self'
https://api.your-secure-proxy.com/;"
  }
}

```

This configuration provides defense-in-depth. Even if a vulnerability were exploited within the extension, this strict CSP would prevent it from exfiltrating data to an unauthorized, malicious domain, as the browser itself would block the connection attempt.<sup>45</sup>

### 3.4. Secure Temporary Storage with `chrome.storage.local`

While `chrome.storage.local` is insecure for secrets, it is the ideal mechanism for the temporary storage of non-sensitive data, such as the analysis results returned from the ML service or user-configurable settings.<sup>38</sup> Unlike

`window.localStorage`, it is accessible from the service worker, persists even if the user clears their browsing data, and is sandboxed to the extension's origin.<sup>38</sup>

The API is asynchronous, so interactions should use promises or `async/await` syntax.

JavaScript

```
// background.js
```

```
// Caching the analysis result
```

```

async function cacheAnalysisResult(result) {
  await chrome.storage.local.set({ 'lastAnalysis': result });
  console.log("Analysis result cached.");
}

```

```
// Retrieving the cached result
```

```
async function getCachedAnalysisResult() {  
  const data = await chrome.storage.local.get(['lastAnalysis']);  
  return data.lastAnalysis;  
}
```

By correctly partitioning responsibilities—using a server-side proxy for secrets and `chrome.storage.local` for state—the extension can achieve a robust and secure data handling architecture.

## Section 4: Implementing Private Information Retrieval (PIR)

The most advanced requirement of this project is the implementation of Private Information Retrieval (PIR). This cryptographic technique enables the extension to fetch specific cookie analysis results from a server without revealing to the server *which* results are being requested. This provides a powerful privacy guarantee, ensuring that the act of querying the data does not leak information about the user's focus or intent. This section will demystify this complex topic, explain the underlying principles, and provide a practical guide to integrating a PIR library into the browser extension.

### 4.1. Conceptual Overview of Private Information Retrieval

At its core, PIR solves the following problem: a client wants to retrieve the  $i$ -th item from a database of  $N$  items held by a server, but the server must not learn the index  $i$ .<sup>50</sup> The naive solution is for the server to send the entire database to the client, who can then pick the

$i$ -th item locally. This provides perfect privacy but is highly inefficient, with communication costs that scale linearly with the database size.<sup>52</sup> The goal of cryptographic PIR is to achieve the same privacy guarantee with sublinear communication costs.

There are two main categories of PIR:

1. **Information-Theoretic PIR (ITPIR):** Provides unconditional privacy but requires multiple, non-colluding servers, which is often an impractical assumption.<sup>52</sup>
2. **Computational PIR (CPIR):** Relies on cryptographic hardness assumptions (like the Learning with Errors problem) and can work with a single server. Its privacy holds against a computationally bounded server. This model is far more practical for real-world web applications.<sup>51</sup>

For this project, the single-server CPIR model is the appropriate choice. The most efficient modern CPIR schemes are built using a special type of cryptography called **Homomorphic Encryption (HE)**.

#### 4.1.1. Single-Server CPIR with Homomorphic Encryption

Homomorphic Encryption allows certain computations to be performed directly on encrypted data (ciphertexts) without decrypting it first.<sup>56</sup> This is the key mechanism that enables single-server PIR. The high-level workflow is as follows <sup>59</sup>:

1. **Client-Side Query Generation:**
  - The client wants to retrieve the item at index  $i=2$  from a database of size  $N=4$ .
  - It creates a "selection vector," which is a vector of zeros with a single one at the desired index:  $\{0, 1, 0, 0\}$ .
  - The client generates a secret key for the HE scheme and encrypts each element of the selection vector, producing a vector of ciphertexts:  $[Enc(0), Enc(1), Enc(0), Enc(0)]$ .
  - This encrypted query is sent to the server. Due to the semantic security of the HE scheme, the server cannot distinguish the encrypted one from the encrypted zeros; the query looks like random noise.
2. **Server-Side Computation:**
  - The server holds its database in plaintext:  $\{DB_1, DB_2, DB_3, DB_4\}$ .
  - It performs a homomorphic "dot product" between the plaintext database and the encrypted query vector. This involves two HE properties:
    - **Homomorphic Multiplication (by a plaintext):** The server multiplies each database item  $DB_j$  by the corresponding encrypted query element  $Enc(q_j)$ . This results in  $Enc(DB_j * q_j)$ . For our example, this yields  $\{Enc(DB_1 * 0), Enc(DB_2 * 1), Enc(DB_3 * 0), Enc(DB_4 * 0)\}$ , which simplifies to  $\{Enc(0), Enc(DB_2), Enc(0), Enc(0)\}$ .
    - **Homomorphic Addition:** The server adds all the resulting ciphertexts together. The sum of the ciphertexts is a single ciphertext that encrypts

the sum of the plaintexts:  $\text{Enc}(0 + 0 + \text{DB\_2} + 0) = \text{Enc}(\text{DB\_2})$ .

- The server sends this single, final ciphertext back to the client. The server has successfully isolated the desired data item and encrypted it for the client, all without ever knowing which item was selected.

### 3. **Client-Side Decryption:**

- The client receives the single ciphertext  $\text{Enc}(\text{DB\_2})$ .
- Using its secret key, it decrypts the result to recover the plaintext data  $\text{DB\_2}$ .

## 4.2. Practical Implementation in the Browser: Libraries and Technologies

The complex mathematics behind homomorphic encryption is computationally intensive. Executing these operations in standard JavaScript would be prohibitively slow, rendering the application unusable.<sup>60</sup> The solution is to leverage

**WebAssembly (WASM)**, a low-level binary format that allows code written in high-performance languages like Rust and C++ to run in the browser at near-native speeds.<sup>62</sup> Modern PIR libraries are therefore typically written in a systems language and provide WASM builds for web compatibility.

A survey of available and practical libraries reveals several strong candidates:

- **Blyss SDK:** A modern, open-source SDK specifically designed for private data access using HE. Its core cryptographic logic is implemented in Rust (based on advanced schemes like SpiralPIR) and is exposed to JavaScript/TypeScript via a WASM bridge. It provides a user-friendly client library and an open-source server, making it an excellent choice for this project.<sup>65</sup>
- **libepir:** A client-side library for EllipticPIR, which uses EC-ElGamal encryption as its cryptographic basis. It also offers a WebAssembly build for browser integration, making it another viable option.<sup>66</sup>

These libraries abstract away the complex cryptography, providing a higher-level API for developers to perform private queries.

## 4.3. Integrating a PIR Library: A Step-by-Step Guide

Using the Blyss SDK as the primary example, the integration process into the extension's service worker would follow these steps:

### 1. Installation and Setup:

- Install the SDK into the project using npm: `npm install @blyss/sdk`.
- If using a bundler like Webpack or Rollup (common for extension development), import the client into the service worker script: `import { Client } from '@blyss/sdk';`

### 2. Handling the Offline/Online Model:

A critical aspect of modern, high-performance PIR schemes like SimplePIR (which inspires the Blyss SDK's underlying technology) is the use of an "offline/online" or "stateful" model.<sup>55</sup>

- **Offline Phase:** Before any queries can be made, the server performs a one-time, computationally expensive preprocessing of the database to generate a public "hint".<sup>67</sup>
- **Hint Download:** The client (our extension) must download this hint. The hint can be quite large—for example, SimplePIR requires a 121 MB hint for a 1 GB database.<sup>67</sup> This is a significant one-time communication cost.
- **Online Phase:** Once the hint is downloaded and stored, the client can make an unlimited number of extremely fast online queries.

The extension's architecture must account for this. The service worker should check `chrome.storage.local` for the hint upon startup. If it's not present, it should initiate the download and store it. The UI should reflect this one-time setup process to the user.

### 3. Client-Side Operations (Code Example):

JavaScript

```
// background.js
import { Client } from '@blyss/sdk';

const BLYSS_API_KEY = 'YOUR_API_KEY_HERE'; // This should be for the Blyss cloud service or a
self-hosted instance
const BLYSS_BUCKET_NAME = 'cookie-analysis-results';

let blyssClient;

async function initializeBlyssClient() {
```

```

    if (!blyssClient) {
      blyssClient = await Client.new(BLYSS_API_KEY);
    }
    return blyssClient;
  }

  // Function to perform a private read
  async function privateFetchCookieResult(index) {
    const client = await initializeBlyssClient();

    try {
      // The SDK handles the complex query generation and decryption
      const result = await client.privateRead(BLYSS_BUCKET_NAME, index);

      // The result is a Uint8Array, which needs to be decoded
      const textDecoder = new TextDecoder();
      const resultString = textDecoder.decode(result);

      return JSON.parse(resultString);
    } catch (error) {
      console.error("PIR failed:", error);
      throw error;
    }
  }
}

```

#### 4. Server-Side Component:

PIR is a two-party protocol. The extension acts as the client, but it needs a compatible server that holds the data and can process the homomorphic queries. The developer would need to either use the Blyss cloud service or run the open-source Blyss Rust server, populating its database with the analyzed cookie data from the ML model.<sup>65</sup>

The choice of a PIR scheme is a significant engineering decision involving a multi-dimensional trade-off. It is not simply a matter of picking the "fastest" library. Schemes like SimplePIR offer incredible server-side computational throughput but at the cost of a large initial hint download.<sup>67</sup> Other schemes might offer lower communication overhead at the cost of higher server CPU usage, which directly impacts operational costs.<sup>71</sup> Furthermore, standard PIR schemes do not protect against a malicious server that returns incorrect or manipulated data. For that, more



advanced and computationally expensive

**Authenticated PIR (APIR)** schemes are required, which can be 30-100x more costly but provide a guarantee of data integrity.<sup>73</sup> The developer must weigh these factors—latency, bandwidth, server cost, and security guarantees—to select the protocol that best aligns with the project's specific constraints and threat model.

**Table 4.1: Comparison of Practical JavaScript-Compatible PIR Libraries**

| Library          | Underlying Scheme     | Security Basis                   | Key Features                                                                           | Browser Integration              | Performance Characteristics                                                                                                                 |
|------------------|-----------------------|----------------------------------|----------------------------------------------------------------------------------------|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Blyss SDK</b> | SpiralPIR / DoublePIR | Ring Learning With Errors (RLWE) | Offline/Online model with hint, batching support, cloud service or self-hosted server. | WebAssembly (from Rust core)     | Optimized for high server throughput and low-latency online queries, at the cost of a significant one-time hint download.                   |
| <b>libepir</b>   | EllipticPIR           | EC-ElGamal (DDH-based)           | Client-side library for query generation and response decryption.                      | WebAssembly (from C++/Rust core) | Relies on elliptic curve cryptography; performance trade-offs differ from lattice-based schemes. Focuses on the client-side implementation. |

## Section 5: Synthesis and End-to-End Implementation

This final technical section synthesizes the architectural principles, API interactions, security patterns, and cryptographic protocols discussed previously into a cohesive, end-to-end proof-of-concept. We will present the complete project structure and annotated source code, illustrating the full data lifecycle—from the user's initial click in the popup to the private retrieval of analyzed cookie data.

### 5.1. Project Structure and File Organization

A well-organized project structure is crucial for maintainability. The final extension directory will be laid out as follows:

```
cookie-analyzer-extension/  
├── icons/  
│   ├── icon16.png  
│   ├── icon48.png  
│   └── icon128.png  
├── popup/  
│   ├── popup.html  
│   ├── popup.css  
│   └── popup.js  
├── background.js  
├── manifest.json  
├── node_modules/ # (Or vendor folder for bundled libraries)  
└── @blyss/sdk/
```

- **icons/**: Contains the extension's icons for various resolutions.
- **popup/**: Holds all files related to the user-facing popup interface.<sup>3</sup>
- **background.js**: The service worker, which serves as the extension's core logic controller.<sup>3</sup>

- **manifest.json**: The central configuration file that defines the extension's properties and permissions.<sup>4</sup>
- **node\_modules/**: Contains the installed PIR library (e.g., Blyss SDK), which will be bundled with the extension.

## 5.2. The Complete Data Flow: From Browser to Private Retrieval

The following code provides a functional skeleton for the extension, demonstrating the complete, multi-stage process.

### 5.2.1. manifest.json

This manifest declares all necessary permissions and registers the core components.

JSON

```
{
  "manifest_version": 3,
  "name": "Privacy-Preserving Cookie Analyzer",
  "version": "1.0",
  "description": "Fetches cookies, sends them for analysis, and retrieves results privately.",
  "permissions": [
    "cookies",
    "storage"
  ],
  "host_permissions": [
    "<all_urls>",
    "https://api.your-secure-proxy.com/"
  ],
  "background": {
    "service_worker": "background.js",
  }
}
```

```

    "type": "module"
  },
  "action": {
    "default_popup": "popup/popup.html",
    "default_icon": {
      "16": "icons/icon16.png",
      "48": "icons/icon48.png",
      "128": "icons/icon128.png"
    }
  },
  "content_security_policy": {
    "extension_pages": "script-src 'self' 'wasm-unsafe-eval'; object-src 'self';"
  }
}

```

*Note: 'wasm-unsafe-eval' is often required in the CSP for WebAssembly to function correctly, as some WASM compilation targets need it for performance optimizations. This should be verified with the specific PIR library's documentation.*

### 5.2.2. popup/popup.html and popup.js (The User Interface)

The popup provides the user interface to trigger the analysis and view results.

HTML

```

<!DOCTYPE html>
<html>
<head>
  <title>Cookie Analyzer</title>
  <link rel="stylesheet" href="popup.css">
</head>
<body>
  <h1>Cookie Analysis Tool</h1>
  <button id="analyzeBtn">Analyze All Cookies</button>
  <hr>

```

```

<h2>Retrieve Analysis Result</h2>
<input type="number" id="resultIndex" placeholder="Enter result index (e.g., 0)">
<button id="retrieveBtn">Retrieve Privately</button>
<pre id="resultArea"></pre>
<script src="popup.js"></script>
</body>
</html>

```

JavaScript

```

// popup.js
document.addEventListener('DOMContentLoaded', () => {
  const analyzeBtn = document.getElementById('analyzeBtn');
  const retrieveBtn = document.getElementById('retrieveBtn');
  const resultIndexInput = document.getElementById('resultIndex');
  const resultArea = document.getElementById('resultArea');

  // 1. Trigger the analysis
  analyzeBtn.addEventListener('click', () => {
    resultArea.textContent = 'Starting analysis...';
    chrome.runtime.sendMessage({ command: 'startAnalysis' }, (response) => {
      if (response.success) {
        resultArea.textContent = `Analysis complete. ${response.count} cookies sent.`;
      } else {
        resultArea.textContent = `Error: ${response.error}`;
      }
    });
  });

  // 6. Trigger the private retrieval
  retrieveBtn.addEventListener('click', () => {
    const index = parseInt(resultIndexInput.value, 10);
    if (isNaN(index)) {
      resultArea.textContent = 'Please enter a valid index.';
      return;
    }
    resultArea.textContent = `Privately retrieving result for index ${index}...`;
  });
});

```

```

chrome.runtime.sendMessage({ command: 'privateRetrieve', index: index }, (response) => {
  if (response.success) {
    // 8. Display the decrypted result
    resultArea.textContent = JSON.stringify(response.data, null, 2);
  } else {
    resultArea.textContent = `Error: ${response.error}`;
  }
});
});
});

```

### 5.2.3. background.js (The Core Logic)

The service worker orchestrates the entire process, from data collection to secure communication and private retrieval.

JavaScript

```

// background.js
// Note: Assumes a bundler is used to handle this import.
import { Client } from './node_modules/@blyss/sdk/index.js';

const PROXY_URL = 'https://api.your-secure-proxy.com/analyze';
const BLYSS_API_KEY = 'YOUR_BLYSS_API_KEY';
const BLYSS_BUCKET_NAME = 'cookie-analysis-results';

let blyssClient;

// Initialize the PIR client
async function initializeBlyssClient() {
  if (!blyssClient) {
    // In a real implementation, the hint would be downloaded and cached here.
    // The SDK may handle this internally or require manual management.
    console.log("Initializing PIR Client...");
  }
}

```

```

    blyssClient = await Client.new(BLYSS_API_KEY);
    console.log("PIR Client initialized.");
  }
  return blyssClient;
}

// Main message listener
chrome.runtime.onMessage.addListener((message, sender, sendResponse) => {
  if (message.command === 'startAnalysis') {
    handleStartAnalysis().then(sendResponse);
    return true; // Asynchronous response
  }
  if (message.command === 'privateRetrieve') {
    handlePrivateRetrieve(message.index).then(sendResponse);
    return true; // Asynchronous response
  }
});

async function handleStartAnalysis() {
  try {
    // 2. Cookie Collection
    const cookies = await chrome.cookies.getAll({});

    // 3. Secure Submission to Proxy
    const response = await fetch(PROXY_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(cookies)
    });

    if (!response.ok) throw new Error(`Proxy server error: ${response.statusText}`);

    // 4. Backend Processing (happens on the server)
    // The proxy will forward to ML, get results, and store them in the PIR DB.

    console.log("Data sent to proxy for analysis and PIR storage.");
    return { success: true, count: cookies.length };
  } catch (error) {
    console.error('Analysis failed:', error);
  }
}

```

```

    return { success: false, error: error.message };
  }
}

async function handlePrivateRetrieve(index) {
  try {
    // 5. PIR Initialization (if not already done)
    const client = await initializeBlyssClient();

    // 7. Generate query, send to server, and get response
    console.log(`Performing private read for index ${index}...`);
    const encryptedResult = await client.privateRead(BLYSS_BUCKET_NAME, index);

    // 8. Decrypt and decode the result
    const textDecoder = new TextDecoder();
    const resultString = textDecoder.decode(encryptedResult);
    const data = JSON.parse(resultString);

    console.log("Private retrieval successful.");
    return { success: true, data: data };
  } catch (error) {
    console.error('Private retrieval failed:', error);
    return { success: false, error: error.message };
  }
}

```

This end-to-end implementation demonstrates a clear separation of concerns. The popup handles only UI, while the service worker manages all complex, asynchronous, and long-running tasks. The performance of this system will be defined by two distinct phases. The initial setup, particularly the one-time download of the PIR hint, could be a significant performance bottleneck, potentially taking several seconds or more depending on network conditions and hint size.<sup>67</sup> The UI must be designed to handle this gracefully, perhaps with a progress indicator, to manage user expectations. Once this initialization is complete, subsequent online PIR queries are designed to be extremely fast, providing a responsive experience for data retrieval.<sup>75</sup>

### 5.3. Debugging in the Manifest V3 Ecosystem



Debugging an MV3 extension is more complex than a monolithic web page because its components run in separate, isolated contexts. A developer must know where to look to find the logs and errors for each part of the extension <sup>4</sup>:

- **Service Worker:** The console logs, network requests, and errors for the service worker are found by navigating to `chrome://extensions`, enabling "Developer mode," finding the extension, and clicking the "service worker" link. This opens a dedicated DevTools instance for the background context.
- **Popup:** To inspect the popup, right-click on the extension's toolbar icon and select "Inspect popup." This opens a DevTools instance scoped to the popup's HTML document. This window will close automatically when the popup closes.
- **Content Scripts:** The logs and errors for a content script appear in the DevTools console of the web page it has been injected into (accessible via F12 or right-click -> "Inspect" on the page).

Mastering this multi-context debugging workflow is essential for efficiently identifying and resolving issues in an MV3 extension.

## Section 6: Conclusion and Strategic Recommendations

This report has provided a comprehensive technical blueprint for developing a sophisticated, privacy-preserving browser extension for cookie analysis. The proposed architecture leverages the modern, event-driven model of Manifest V3, incorporates robust security practices for API communication, and integrates the cutting-edge cryptographic protocol of Private Information Retrieval. The successful implementation of this project hinges on a clear understanding of the architectural decisions and inherent trade-offs discussed.

### 6.1. Summary of Architectural Decisions

The recommended architecture is built upon three foundational pillars, each addressing a core requirement of the project:

1. **An Event-Driven, Stateless MV3 Structure:** The extension's logic must be centered around the ephemeral service worker. This necessitates a stateless design where all persistent data is managed via the chrome.storage API, and all components (popup, service worker) communicate through asynchronous messaging. This is the most efficient and secure way to build for the modern extension platform.
2. **A Mandatory Server-Side Proxy for API Security:** The API key for the external ML service must never be exposed on the client side. The implementation of a server-side proxy is a non-negotiable security requirement to protect credentials, manage access, and prevent API abuse.
3. **An Offline/Online Model for Practical PIR:** To achieve acceptable performance for Private Information Retrieval, the extension must adopt a stateful, offline/online protocol. This involves a significant one-time cost to download a server-generated "hint," which is then used to enable fast, low-latency online queries.

## 6.2. Balancing Privacy, Security, and Performance

Throughout this analysis, a recurring theme has been the trade-offs between privacy, security, and performance. The decision to use PIR, for example, provides a powerful query privacy guarantee but introduces substantial complexity and performance considerations (both in computation and communication) compared to a standard database query. Similarly, implementing a server-side proxy and a strict Content Security Policy adds development overhead but is essential for a secure system.

A strategic, phased implementation is highly recommended:

- **Phase 1: Core Functionality.** First, build and thoroughly test the core extension for cookie collection and analysis via the server-side proxy, without the PIR component. This will validate the data pipeline and the ML model's effectiveness.
- **Phase 2: PIR Integration.** Once the core functionality is stable, integrate the Private Information Retrieval layer. This allows the development team to focus on the unique challenges of PIR—such as managing the hint and integrating the WASM library—without simultaneously debugging the data collection and analysis pipeline.

### 6.3. Future Directions

The architecture described in this report provides a strong foundation that can be extended with even more advanced features in the future:

- **Authenticated Private Information Retrieval (APIR):** The current PIR model protects client privacy but does not guarantee the integrity of the server's response. A malicious or compromised server could return incorrect data. Future work could involve implementing an APIR scheme, which allows the client to cryptographically verify that the server's response is authentic and consistent with the true database, albeit at a higher computational cost.<sup>73</sup>
- **Batch PIR:** If the use case evolves to require fetching multiple analysis results at once, batch PIR protocols can offer significant efficiency gains. These schemes are designed to retrieve multiple database items in a single query, with communication and computation costs that are substantially lower than performing multiple individual queries.<sup>77</sup>
- **Cross-Browser Compatibility:** While this report focuses on the Chrome extension ecosystem, the WebExtensions API is largely compatible with other browsers like Firefox.<sup>2</sup> The core logic of the service worker, cookies API calls, and PIR integration would likely require minimal changes to be ported, primarily involving adjustments to the manifest.json file to accommodate vendor-specific keys.<sup>5</sup>

By following the detailed guidance and architectural patterns outlined in this report, a development team can successfully build a powerful, secure, and privacy-enhancing tool that meets the complex requirements of the initial query.

### Works cited

1. Extensions / Manifest V3 - Chrome for Developers, accessed August 9, 2025, <https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3>
2. Browser extensions - MDN Web Docs - Mozilla, accessed August 9, 2025, <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions>
3. Building persistent Chrome Extension using Manifest V3 | by Rahul Negi - Medium, accessed August 9, 2025, <https://rahulnegi20.medium.com/building-persistent-chrome-extension-using-manifest-v3-198000bf1db6>
4. Chrome Extensions For Beginners. Getting started with manifest v3 - Jimmy Lam, accessed August 9, 2025, <https://jl978.medium.com/chrome-extensions-for-beginners-46019a826cd6>
5. Your first extension - Mozilla | MDN, accessed August 9, 2025,

[https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Your\\_first\\_WebExtension](https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Your_first_WebExtension)

6. Permissions | Chrome for Developers, accessed August 9, 2025, <https://developer.chrome.com/docs/extensions/reference/permissions-list>
7. 6 Understanding Chrome Extensions Permissions | by M2K Developments - Medium, accessed August 9, 2025, <https://m2kdevelopments.medium.com/6-understanding-chrome-extensions-permissions-04edb197df42>
8. Declare permissions | Chrome Extensions, accessed August 9, 2025, <https://developer.chrome.com/docs/extensions/develop/concepts/declare-permissions>
9. Extensions - Chrome for Developers, accessed August 9, 2025, <https://developer.chrome.com/docs/extensions/develop>
10. Background user scripts in Manifest V3? - Google Groups, accessed August 9, 2025, <https://groups.google.com/a/chromium.org/g/chromium-extensions/c/q9H8KwFLkMs>
11. Manage events with background scripts | Manifest V2 - Chrome for Developers, accessed August 9, 2025, <https://developer.chrome.com/docs/extensions/mv2/background-pages>
12. Chrome Extensions: Content Scripts vs. Background Scripts | by Vanessa Jimenez | Medium, accessed August 9, 2025, [https://medium.com/@vanessajimenez\\_85032/chrome-extensions-content-scripts-vs-background-scripts-7bbd01f9dbe6](https://medium.com/@vanessajimenez_85032/chrome-extensions-content-scripts-vs-background-scripts-7bbd01f9dbe6)
13. Content scripts - Mozilla | MDN, accessed August 9, 2025, [https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Content\\_scripts](https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Content_scripts)
14. Chrome Extension — When to use content scripts and injected scripts | by Radu Diță, accessed August 9, 2025, <https://radu-dita.medium.com/chrome-extension-when-to-use-content-scripts-and-injected-scripts-238563dce8af>
15. Background Scripts vs Content Scripts in Chrome Extensions - Stack Overflow, accessed August 9, 2025, <https://stackoverflow.com/questions/12971869/background-scripts-vs-content-scripts-in-chrome-extensions>
16. Tutorial on how to build a Chrome Extension using Manifest V3 : r/typescript - Reddit, accessed August 9, 2025, [https://www.reddit.com/r/typescript/comments/15pmdv2/tutorial\\_on\\_how\\_to\\_build\\_a\\_chrome\\_extension\\_using/](https://www.reddit.com/r/typescript/comments/15pmdv2/tutorial_on_how_to_build_a_chrome_extension_using/)
17. Chrome Extensions & Manifest v3 with Google DevRel Oliver Dunk - YouTube, accessed August 9, 2025, <https://www.youtube.com/watch?v=TO1Ypwr2Pzw>
18. chrome.cookies | API | Chrome for Developers, accessed August 9, 2025, <https://developer.chrome.com/docs/extensions/reference/api/cookies>
19. API reference - Chrome for Developers, accessed August 9, 2025, <https://developer.chrome.com/docs/extensions/reference/api>

20. cookies - Mozilla | MDN, accessed August 9, 2025,  
<https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/cookies>
21. Cookies API - The Chromium Projects, accessed August 9, 2025,  
<https://www.chromium.org/developers/design-documents/extensions/proposed-changes/apis-under-development/proposal-chrome-extensions-cookies-api/>
22. Chrome Extension - get all cookies for a specific tab - is it correct? - Stack Overflow, accessed August 9, 2025,  
<https://stackoverflow.com/questions/75426272/chrome-extension-get-all-cookies-for-a-specific-tab-is-it-correct>
23. cookies.getAll() - Mozilla - MDN Web Docs, accessed August 9, 2025,  
<https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/cookies/getAll>
24. cookies.getAll() - IGM, accessed August 9, 2025,  
<https://www-igm.univ-mlv.fr/~forax/MDN/developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/cookies/getAll.html>
25. cookies.getAllCookieStores() - Mozilla - MDN Web Docs, accessed August 9, 2025,  
<https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/cookies/getAllCookieStores>
26. cookies.OnChangedCause - Mozilla - MDN Web Docs, accessed August 9, 2025,  
<https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/cookies/OnChangedCause>
27. cookies.onChanged, accessed August 9, 2025,  
<https://frost.cs.uchicago.edu/ref/JavaScript/developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/cookies/onChanged.html>
28. cookies.onChanged - Mozilla - MDN Web Docs, accessed August 9, 2025,  
<https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/cookies/onChanged>
29. HTTP cookie - Wikipedia, accessed August 9, 2025,  
[https://en.wikipedia.org/wiki/HTTP\\_cookie](https://en.wikipedia.org/wiki/HTTP_cookie)
30. Using HTTP cookies - MDN Web Docs - Mozilla, accessed August 9, 2025,  
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies>
31. Cookies, document.cookie, accessed August 9, 2025,  
<https://fa.javascript.info/cookie>
32. Cookie attributes - Privacy Sandbox, accessed August 9, 2025,  
<https://privacysandbox.google.com/cookies/basics/cookie-attributes>
33. What does Priority=High mean in the Set-Cookie header? - Stack Overflow, accessed August 9, 2025,  
<https://stackoverflow.com/questions/19792038/what-does-priority-high-mean-in-the-set-cookie-header>
34. View, add, edit, and delete cookies | Chrome DevTools | Chrome for ..., accessed August 9, 2025, <https://developer.chrome.com/docs/devtools/application/cookies>
35. draft-west-cookie-priority-00 - A Retention Priority Attribute for HTTP Cookies, accessed August 9, 2025,

- <https://datatracker.ietf.org/doc/draft-west-cookie-priority/>
36. Best practices for managing API keys | Authentication | Google Cloud, accessed August 9, 2025, <https://cloud.google.com/docs/authentication/api-keys-best-practices>
  37. Best practices for securely using API keys - Google Help, accessed August 9, 2025, <https://support.google.com/googleapi/answer/6310037?hl=en>
  38. chrome.storage | Reference - Chrome for Developers, accessed August 9, 2025, <https://developer.chrome.com/docs/extensions/mv2/reference/storage>
  39. When Not to Use Local Storage: Risks, Examples, and Secure Alternatives - Meenu Matharu, accessed August 9, 2025, <https://meenumatharu.medium.com/when-not-to-use-local-storage-risks-examples-and-secure-alternatives-de541fed56d2>
  40. Stay secure | Chrome Extensions | Chrome for Developers, accessed August 9, 2025, <https://developer.chrome.com/docs/extensions/develop/security-privacy/stay-secure>
  41. API Security Best Practices | Curity, accessed August 9, 2025, <https://curity.io/resources/learn/api-security-best-practices/>
  42. REST Security - OWASP Cheat Sheet Series, accessed August 9, 2025, [https://cheatsheetseries.owasp.org/cheatsheets/REST\\_Security\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html)
  43. Fetch Data in Chrome Extension V3 | by bitbug | JavaScript in Plain ..., accessed August 9, 2025, <https://javascript.plainenglish.io/fetch-data-in-chrome-extension-v3-2b73719ffc0e>
  44. Cross-Origin Resource Sharing (CORS) - HTTP - MDN Web Docs, accessed August 9, 2025, <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>
  45. Content Security Policy - Mozilla | MDN, accessed August 9, 2025, [https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Content\\_Security\\_Policy](https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Content_Security_Policy)
  46. Content-Security-Policy (CSP) Header Quick Reference, accessed August 9, 2025, <https://content-security-policy.com/>
  47. Using Content Security Policy (CSP) to control which resources can be run - Microsoft Edge Developer documentation, accessed August 9, 2025, <https://learn.microsoft.com/en-us/microsoft-edge/extensions/developer-guide/csp>
  48. Storage and cookies | Chrome Extensions | Chrome for Developers, accessed August 9, 2025, <https://developer.chrome.com/docs/extensions/develop/concepts/storage-and-cookies>
  49. chrome.storage | API - Chrome for Developers, accessed August 9, 2025, <https://developer.chrome.com/docs/extensions/reference/api/storage>
  50. ELI15: Private Information Retrieval - Educated Guesswork, accessed August 9, 2025, <https://educatedguesswork.org/posts/pir/>
  51. Unlocking Private Information Retrieval - Number Analytics, accessed August 9,



- 2025,  
<https://www.numberanalytics.com/blog/private-information-retrieval-computational-number-theory>
52. Private information retrieval - Wikipedia, accessed August 9, 2025,  
[https://en.wikipedia.org/wiki/Private\\_information\\_retrieval](https://en.wikipedia.org/wiki/Private_information_retrieval)
  53. What is the difference between PIR and OT? [duplicate] - Cryptography Stack Exchange, accessed August 9, 2025,  
<https://crypto.stackexchange.com/questions/64577/what-is-the-difference-between-pir-and-ot>
  54. Private Information Retrieval and Its Applications: An Introduction, Open Problems, Future Directions - arXiv, accessed August 9, 2025,  
<https://arxiv.org/pdf/2304.14397>
  55. How Practical is Single-Server Private Information Retrieval? - ETH Zürich, accessed August 9, 2025,  
[https://ethz.ch/content/dam/ethz/special-interest/infk/inst-infsec/appliedcrypto/education/theses/semester-project\\_sophia-artioli.pdf](https://ethz.ch/content/dam/ethz/special-interest/infk/inst-infsec/appliedcrypto/education/theses/semester-project_sophia-artioli.pdf)
  56. Private Information Retrieval: A Deep Dive - Number Analytics, accessed August 9, 2025,  
<https://www.numberanalytics.com/blog/private-information-retrieval-deep-dive-technical-aspects>
  57. Combining Machine Learning and Homomorphic Encryption in the Apple Ecosystem, accessed August 9, 2025,  
<https://machinelearning.apple.com/research/homomorphic-encryption>
  58. Linearly Homomorphic Encryption and Private Information Retrieval - MIT, accessed August 9, 2025, <https://65610.csail.mit.edu/2024/lec/I05-pir.pdf>
  59. Private information retrieval using homomorphic encryption (explained from scratch), accessed August 9, 2025,  
<https://blintzbase.com/posts/pir-and-fhe-from-scratch/>
  60. JavaScript performance optimization - Learn web development | MDN, accessed August 9, 2025,  
[https://developer.mozilla.org/en-US/docs/Learn\\_web\\_development/Extensions/Performance/JavaScript](https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Performance/JavaScript)
  61. 8 JavaScript Performance Tips I Wish I Knew Sooner | by Adarsh Rai - Medium, accessed August 9, 2025,  
<https://medium.com/@adarshrai3011/8-javascript-performance-tips-i-wish-i-knew-sooner-42078c542247>
  62. WASM Tutorial - Marco Selvatici, accessed August 9, 2025,  
[https://marcoselvatici.github.io/WASM\\_tutorial/](https://marcoselvatici.github.io/WASM_tutorial/)
  63. Emscripting a C library to Wasm | Articles - web.dev, accessed August 9, 2025,  
<https://web.dev/articles/emscripting-a-c-library>
  64. Web Assembly Tutorial: Part 1 - by Bpax - Medium, accessed August 9, 2025,  
<https://medium.com/@lcbypain/web-assembly-tutorial-part-1-5d513c4a7044>
  65. blyssprivacy/sdk: SDK for private data access, built on ... - GitHub, accessed August 9, 2025, <https://github.com/blyssprivacy/sdk>
  66. EllipticPIR/libepir: EllipticPIR client library for C / C++ / Rust ... - GitHub, accessed

- August 9, 2025, <https://github.com/EllipticPIR/libepir>
67. One Server for the Price of Two: Simple and Fast Single-Server Private Information Retrieval - USENIX, accessed August 9, 2025, [https://www.usenix.org/system/files/sec23summer\\_27-henzinger-prepub.pdf](https://www.usenix.org/system/files/sec23summer_27-henzinger-prepub.pdf)
  68. FrodoPIR: Simple, Scalable, Single-Server Private Information Retrieval - Privacy Enhancing Technologies Symposium, accessed August 9, 2025, <https://petsymposium.org/popets/2023/popets-2023-0022.pdf>
  69. Understanding SimplePIR & DoublePIR - Yue's Notes, accessed August 9, 2025, <https://ofuro.me/posts/simple-double/>
  70. The reference implementation of SimplePIR and DoublePIR, accompanying the paper "One Server for the Price of Two: Simple and Fast Single-Server Private Information Retrieval" (USENIX Security 2023). - GitHub, accessed August 9, 2025, <https://github.com/ahenzinger/simplepir>
  71. Communication-Computation Trade-offs in PIR | USENIX, accessed August 9, 2025, <https://www.usenix.org/conference/usenixsecurity21/presentation/ali>
  72. Communication-Computation Trade-offs in PIR - Google Research, accessed August 9, 2025, <https://research.google/pubs/communicationcomputation-trade-offs-in-pir/>
  73. Authenticated private information retrieval - USENIX, accessed August 9, 2025, <https://www.usenix.org/conference/usenixsecurity23/presentation/colombo>
  74. Authenticated private information retrieval - USENIX, accessed August 9, 2025, <https://www.usenix.org/system/files/sec23fall-prepub-78-colombo.pdf>
  75. Henry Corrigan-Gibbs: Simple and Fast Private Information Retrieval - YouTube, accessed August 9, 2025, <https://www.youtube.com/watch?v=Jdzrf3im1gQ>
  76. Authenticated private information retrieval - Kirill Nikitin, accessed August 9, 2025, <https://nikirill.com/files/apir.pdf>
  77. [2205.05211] TreePIR: Efficient Private Retrieval of Merkle Proofs via Tree Colorings with Fast Indexing and Zero Storage Overhead - arXiv, accessed August 9, 2025, <https://arxiv.org/abs/2205.05211>
  78. Batch PIR and Labeled PSI with Oblivious Ciphertext Compression - USENIX, accessed August 9, 2025, <https://www.usenix.org/conference/usenixsecurity24/presentation/bienstock>
  79. Batched Differentially Private Information Retrieval - USENIX, accessed August 9, 2025, <https://www.usenix.org/conference/usenixsecurity22/presentation/albab>
  80. Firefox Extension Workshop | Get help creating & publishing Firefox extensions., accessed August 9, 2025, <https://extensionworkshop.com/>